Execution-Aware Agentic VRP Solver

and Benchmark Studio

8	18	770s	0
SOLVER BACKENDS	TEST SCENARIOS	WINNER RUNTIME	VIOLATIONS

Anubha Parashar · April 2026

What we'll cover

Part 1 · Problem & Vision

Static VRP solvers vs live dispatch reality

Part 2 · The 8 Backends

Algorithm families & complexity analysis

Part 3 · Split-Solve Innovation

Why OR-Tools EA wins · locked-prefix mechanism

Part 4 · Benchmark Results

Per-scenario performance · trade-offs

Part 5 · 18 Scenarios

EAS · RT · GB stress tests

Part 6 · Agentic AI Layer

Gemini · 8 tools · 4 behaviors

Part 7 · Architecture & Code

5 agents · 8,140 lines · system flow

Part 8 · Deployment & Demo

Docker · HF Spaces · live walkthrough

The Problem

Most VRP solvers are designed for static route planning.
Real dispatch is dynamic — and chaotic.

In production reality

- Technicians are mid-route when changes hit
- Drivers call in sick at 8 AM
- Urgent SLA orders inject mid-day
- Customers cancel after dispatch
- Shifts get delayed or cut short
- Service times shift as work expands

What standard solvers do

- Re-solve from scratch every time
- Reroute already-completed stops
- Lose execution state
- Miss SLA windows during re-planning
- No awareness of in-progress work
- Treat every change as a full restart

The Vision

Build a VRP solver that knows execution state and a benchmark that proves it works.

1 · Compare

Side-by-side benchmark of 8 solver families on identical inputs.

Fair, reproducible, and visual.

- Greedy · OSRM · PyVRP
- OR-Tools · OR-Tools EA
- Hybrid · Adaptive · Esri

2 · Preserve

Lock in-progress routes so re-optimization respects committed work.

New innovation: split-solve.

- Locked stops bypass solver
- Vehicle anchored at last stop
- Only open work re-routed

3 · Self-correct

An AI agent that detects violations and triggers re-optimization autonomously.

Gemini 2.0 Flash · 8 tools.

- Natural language interface
- Multi-step reasoning
- Reactive event handling

Live Demo

Hosted on Hugging Face Spaces · pre-loaded with synthetic dataset & benchmark artifacts

🚀 **LIVE STREAMLIT APP**

8 backends pre-computed · 18 scenarios · AI agent ready

https://huggingface.co/spaces/AnubhaParashar/Execution-Aware-Agentic-VRP-Solver-and-Benchmark-Studio

→ Open the link to interact with the live app while presenting

Free Gemini 2.0 Flash API key required for the AI Agent tab · 1,500 req/day

The 8 Solver Backends

🏆 OR-Tools EA ★

Split-solve CP + GLS + greedy fallback
770s · 0 violations · 150/150

OR-Tools

CVRPTW + Guided Local Search
868s · 0 violations · 150/150

Esri ArcGIS

Tabu Search (cloud)
973s · drops 8 orders avg

Adaptive

Multi-restart regret
6,472s · 0 violations · 150/150

Hybrid

Regret + LNS
1,790s · 0 violations · 150/150

Greedy + 2-opt

Nearest-neighbour
22s · 231,070 late min

PyVRP

Hybrid Genetic Search
545s · 134,483 late min

OSRM

Greedy + road distances
4,949s · 245,827 late min

Filter

Production-viable
Only 4 of 8 backends pass

Algorithm Families

Family	Backends	Approach	TW Handling	Production?
Constraint Programming	OR-Tools, OR-Tools EA	Variables + constraints + search	Hard	✓ Yes
Metaheuristics	Hybrid, Adaptive, PyVRP	Iterative improvement	Penalty / Hard	△ Specialized
Greedy Heuristics	Greedy, OSRM	Local decisions, no global view	Cost only	✗ Baseline
Commercial Cloud	Esri ArcGIS	Tabu Search (proprietary)	Soft (drops)	△ Enterprise

All 8 backends follow the same I/O contract: run_*_backend(inputs, config) → bundle

Master Benchmark Table

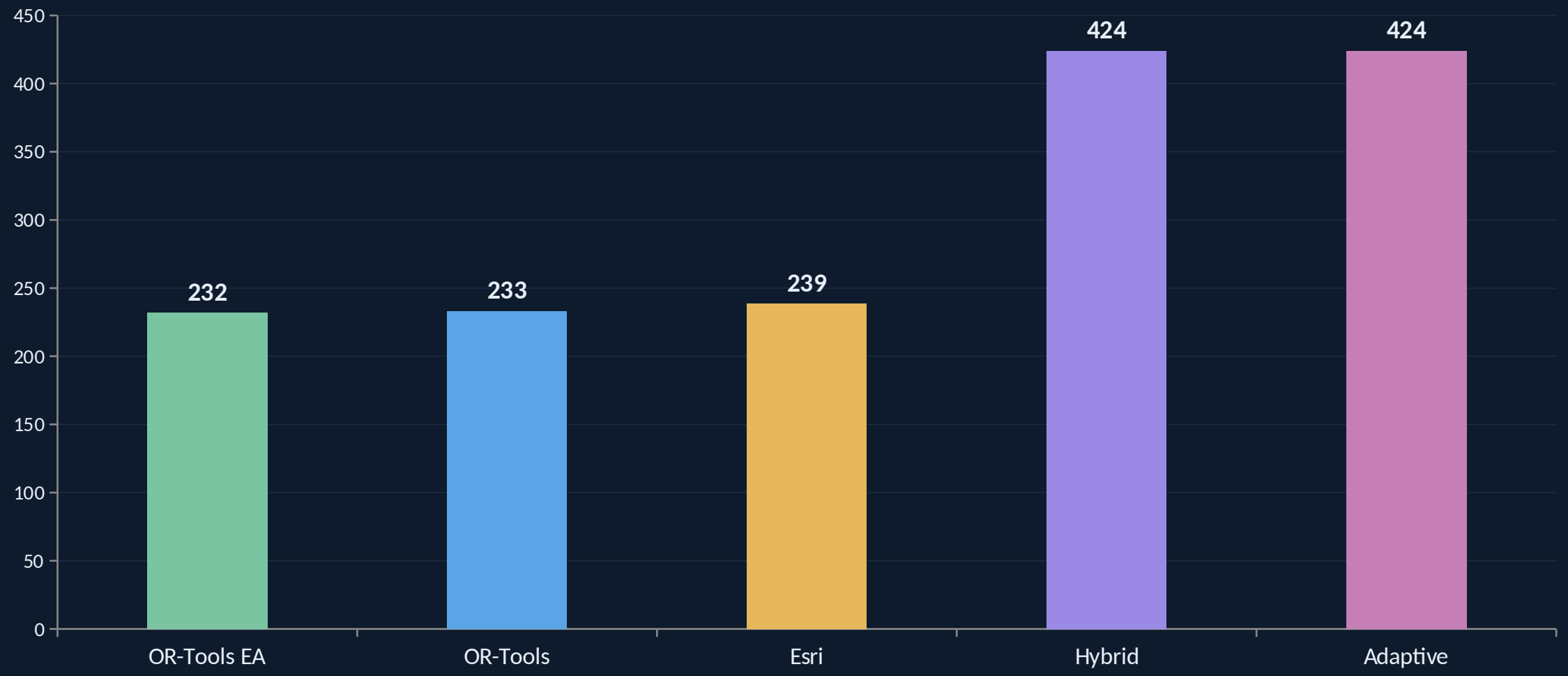
150 orders · 20 routes · 18 scenarios · means

Backend	Asgn	BASE km	EAS km	RT km	GB km	Late	OT	Runtime
OR-Tools EA	150	231.8	232.8	232.1	239.8	0	0	770s
OR-Tools	150	233.2	233.4	234.6	243.2	0	0	868s
Esri	142	238.7	238.8	206.3	240.3	34	0	973s
Adaptive	150	424.1	349.7	364.1	341.6	0	0	6,472s
Hybrid	150	424.1	465.3	462.2	356.1	0	0	1,790s
Greedy	150	394.4	387.1	387.3	391.2	231,070	46,191	22s
PyVRP	150	114.0	111.8	112.5	118.1	134,483	70,615	545s
OSRM	150	519.3	522.5	518.8	509.9	245,827	52,266	4,949s

OR-Tools EA: only backend with 100% assignment + zero violations + sub-1000s runtime

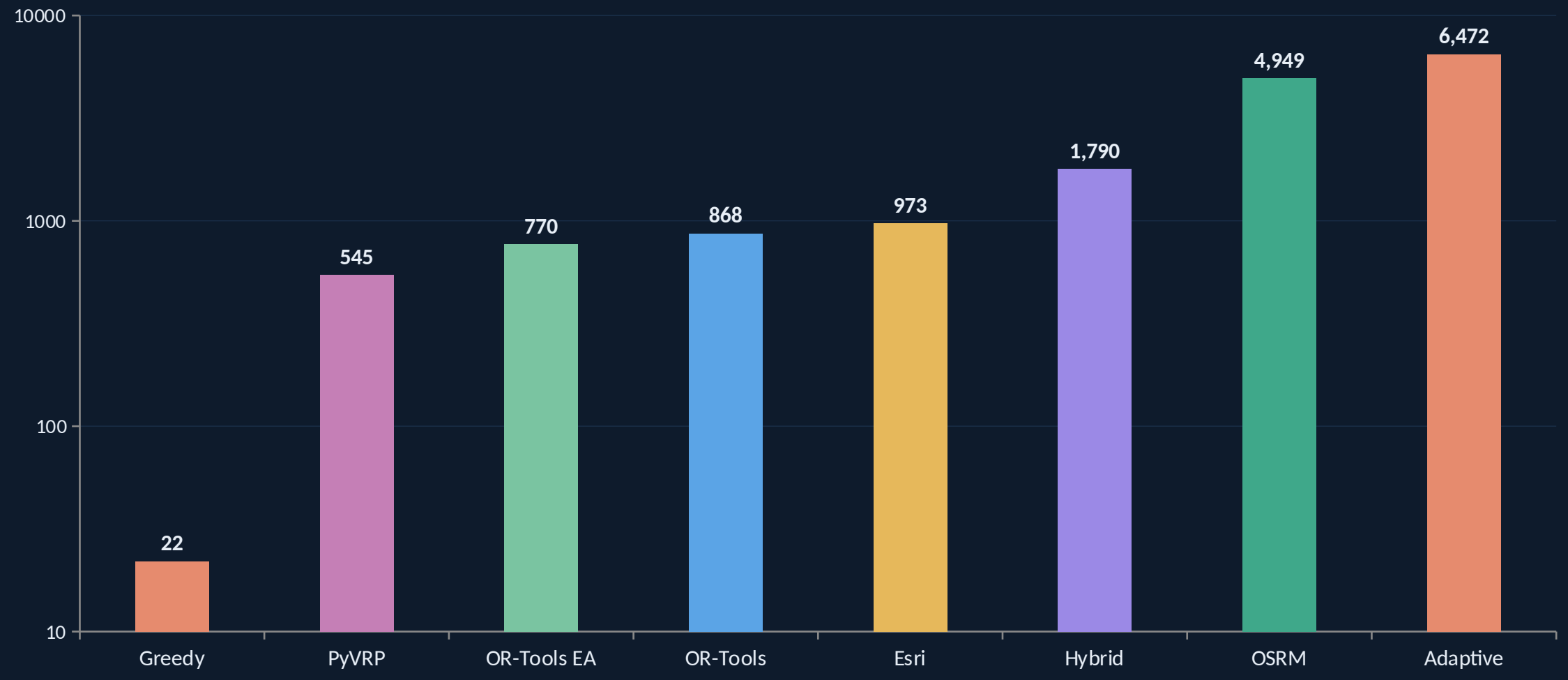
Distance — Zero-Violation Backends

BASE scenario distance in km · lower is better



Runtime — All 18 Scenarios Combined

Total runtime in seconds · log scale



OR-Tools vs OR-Tools EA — Head-to-Head

Metric	OR-Tools (plain)	OR-Tools EA	Improvement
BASE distance	233.2 km	231.8 km	−1.4 km
EAS avg distance	233.4 km	232.8 km	−0.6 km
RT avg distance	234.6 km	232.1 km	−2.5 km
GB avg distance	243.2 km	239.8 km	−3.4 km
Late minutes	0	0	same
Overtime minutes	0	0	same
Total runtime	**868 s**	**770 s**	**−11%**

Same accuracy. 11% faster. Better on every scenario group.

The Split-Solve Innovation

Locked orders never enter the OR-Tools model

✗ Naïve approach

- Submit all 150 orders
- 130 with zero-slack TW constraints
- PATH_CHEAPEST_ARC fails
- 0 orders assigned
- Solver wastes time on infeasibility
- No way to improve from there

✓ Split-solve

- Submit only ~10 open orders
- Locked stops copied verbatim
- Vehicle anchored at last locked stop
- GLS gets full 45s budget
- 150/150 assigned · zero violations
- Greedy fallback catches edge cases

Split-Solve Pipeline

1 Separate

Locked vs open orders by OrderId

2 Anchor

Each vehicle starts at last locked stop

3 Solve

Small OR-Tools model full GLS budget

4 Merge

Locked verbatim + open from solver

5 Fallback

Greedy catch · 100% assignment

```
def _ortools_ea_solve_scenario(scenario, orders, routes, locked_prefixes):
    locked_ids = {oid for prefix in locked_prefixes.values() for oid in prefix.OrderId}
    open_orders = orders[~orders.OrderId.isin(locked_ids)]
    anchors = {r: _anchor_position(r, locked_prefixes[r]) for r in routes}
    solution = ortools_solve(open_orders, routes, anchors, time_limit=45)
    result = _merge_locked_with_solution(locked_prefixes, solution, anchors)
    return _greedy_catch_unassigned(result, open_orders)
```

Locked-Prefix Mechanism

Determines which stops are stable and can be frozen

Detection logic

- changed_order_ids = baseline ⊕ scenario
- impacted_routes = routes containing changes
- fully_locked = routes with zero changes
- partial_lock = stops up to first change

Special cases

- RT_1 (urgent inject) → unlock 2 nearest routes
- RT_6 (sick technician) → drop route entirely
- RT_7 (combined) → multi-route unlock
- Default → preserve all stable stops

```
def _prepare_locked_prefixes(scenario_key, baseline, scenario_orders):
    changed = _changed_order_ids(baseline.orders, scenario_orders)
    impacted = {r for r in baseline.routes if any(s.OrderId in changed for s in r.stops)}
    locked = {r.name: r.stops[:first_impact(r, changed)]
              if r.name in impacted else r.stops
              for r in baseline.routes}
    return locked
```

The 18 Test Scenarios

BASE · 1 scenario

Baseline solve · no mutation applied

EAS · 4 scenarios

Execution-Aware: TimeWindow shift, ServiceTime change, Depot swap, Downstream cascade

RT · 7 scenarios

Real-Time Scheduling

- RT_1: Inject urgent order
- RT_2: Cancellation
- RT_3: ServiceTime ×1.35
- RT_4: Shift delay +90min
- RT_5: Window shrink
- RT_6: Technician sick
- RT_7: Combined real-time

GB · 6 scenarios

Goal-Based Scheduling

- GB_1: Minimize travel
- GB_2: Prefer rated techs
- GB_3: Prefer specialised
- GB_4: Maximise SLA
- GB_5: Reduce overtime
- GB_6: Customer weights

The Agentic AI Layer

Powered by Google Gemini 2.0 Flash · free tier · 1,500 req/day

VRPOrchestrator class

- Wraps Gemini with function-calling
- 8 callable tools (JSON Schema → FunctionDeclaration)
- 4 distinct agentic behaviors
- Max 10 iterations per query
- Stateful chat session
- Tool calls log for full audit trail
- Manual loop (not automatic_function_calling)

Why agentic, not just chat?

- Detects violations autonomously
- Picks config without human input
- Re-runs solver, compares results
- Chains multiple tool calls in one turn
- No 'ask permission' loops
- Real-time event handling
- Self-correcting under disruption

8 Agent Tools

Discovery

- get_scenario_list
  Returns all 18 scenarios
- explain_scenario
  What does this scenario test?

Analysis

- analyze_results
  Summary metrics
- detect_violations
  Late routes + overtime
- compare_backends
  Ranked + winner

Action

- suggest_config
  Recommend overrides
- run_backend
  Live solve
- trigger_reoptimization
  Re-run + diff

System prompt: *'For violation complaints: call detect_violations, then trigger_reoptimization. Chain tool calls — do NOT ask user for permission between calls.'*

4 Agentic Behaviors

1 · Natural language

User: 'Which backend for SLA?'
→ compare_backends → ranked answer
→ analyze_results on top backend → explain

2 · Autonomous re-optimization

User: 'Fix violations in Greedy'
→ detect_violations → suggest_config
→ trigger_reoptimization → before/after diff

3 · Multi-step reasoning

User: 'Run A and B, which is better?'
→ run_backend(A) → run_backend(B)
→ compare_backends → ranked comparison

4 · Real-time reactive

Button: 'Driver sick (RT_6)'
→ explain_scenario → run_backend(OR-Tools EA)
→ analyze_results → recommend re-routing

The Agent Loop

```python
def _run_agent_loop(self, user_message: str) -> str:
    max_iterations = 10
    current_message = user_message

    while iteration < max_iterations:
        response = self._chat_session.send_message(current_message)

        # Collect function calls from response
        fn_calls = [p.function_call for c in response.candidates
                    for p in c.content.parts
                    if hasattr(p, 'function_call') and p.function_call.name]

        if not fn_calls:  # Pure text — agent done
            return extract_text(response)

        # Execute tools and feed results back
        for fn_call in fn_calls:
            result = self._dispatch_tool(fn_call.name, dict(fn_call.args))
            self._tool_calls_log.append({'tool': fn_call.name, 'result': result})
```

System Architecture

8,140 lines across 4 source files

UI Layer · app.py (845 lines)

Streamlit · 6 tabs · sidebar config · auto-load · session state

Domain Layer · base_engine.py (933 lines)

5 agents · scenario engine · haversine · Folium maps

Solver Layer · compare_backends.py (5,438 lines)

8 backends · locked-prefix · OR-Tools EA · Excel export

Agent Layer · vrp_agent.py (960 lines)

Gemini · 8 tools · agentic loop · system prompt

5 domain agents

IntakeAgent

validation

PolicyAgent

weights per goal profile

AssignmentAgent

greedy + 2-opt scoring

ScenarioAgent

18 mutations

ExplainAgent

human-readable actions

Input Data Model

5 CSV files · 40,000 orders · 400 routes · 8 depots

File	Rows	Key Columns
`orders_40000.csv`	40,000	Latitude, Longitude, ServiceTime, TimeWindow, Priority, SLAFlag, AccountId
`routes_400.csv`	400	StartDepotName, EarliestStartTime, MaxOrderCount, OvertimeStartTime, TechnicianRating
`depots_8.csv`	8	Name, Latitude, Longitude
`accounts.csv`	120	AccountId, GoalProfile
`goal_profiles.csv`	4	minimize_travel_time, maximize_sla_adherence, reduce_overtime, prefer_higher_rated

Solve scope per run

150 orders · 20 routes · 1 depot

Distance metric

Haversine at 30 km/h (OSRM uses real roads)

Streamlit UI · 6 Tabs

Overview

Backend run count, scenarios, dataset metrics, summary tables

KPI Comparison

6 Plotly bar charts: Assigned, Routes, Distance, Travel, Late, Runtime

Map Explorer

Interactive Folium map · per-route stops · sequence polylines · ETA popups

Scenario Tables

Per-backend × per-scenario data: Summary, Actions, Routes, Stops

Export

Multi-sheet XLSX download · 4 sheets per backend × 8 = 32 sheets

🤖 AI Agent

Chat · 8 quick actions · 4 reactive event buttons · tool calls log

BackendConfig — 28 Parameters

All exposed in the Streamlit sidebar

Core (9)

- speed_kmph = 35.0
- use_two_opt = True
- strict_skill_bias = True
- random_seed = 42
- max_orders = 350
- max_routes = 24
- depot_focus = None
- sample_by_priority = True
- fast_mode = True

Hybrid/Adaptive (14)

- candidate_routes = 10
- frontier_size = 28
- local_search_passes = 4
- max_hard_late_min = 30
- shift_buffer = 15
- route_activation = 42.0
- compact_passes = 3
- global_relocate = 3
- require_zero_late = True

Solver-specific (5)

OR-Tools (2):
- ortools_max_runtime = 5
- ortools_allow_drop = True

PyVRP (2):
- pyvrp_max_runtime = 5
- pyvrp_allow_drop = False

OSRM (6) · Esri (4) · ...

Complexity Analysis

Backend	Assignment	Local Search	Total
Greedy	$O(N \times R)$	$O(R \times M^2)$ per pass	$O(NR + RM^2)$
OSRM	$O(N \times R \times HTTP)$	$O(RM^2 \times HTTP)$	$O((NR + RM^2) \times HTTP)$
PyVRP	$O(N^2)$ per gen	C++ moves	$O(G \times N^2)$
OR-Tools	$O(N \times R)$	$O(t \times N \times R)$	$O(t \times N \times R)$
OR-Tools EA	**$O(K \times R)$**	**$O(t \times K \times R)$**	**$O(t \times K \times R)$, $K << N$**
Hybrid	$O(N \times R \times frontier)$	$O(NR \times passes)$	$O(NR \times passes)$
Adaptive	$K_restarts \times Hybrid$	$K \times Hybrid$	$K \times O(NR \times passes)$

OR-Tools EA submits K = ~10 open orders instead of N = 150. Model is 10-15× smaller.

Deployment Path

1 Local dev

Run benchmarks
save artifacts

2 Docker build

HF Spaces
base image

3 Push to HF

git lfs for
7MB CSV

4 Set secrets

GOOGLE_API_KEY
in Settings

5 Live

Public HTTPS
endpoint

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y --no-install-recommends build-essential
RUN useradd -m -u 1000 user
USER user
WORKDIR /home/user/app
COPY --chown=user requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=user . .
EXPOSE 7860
CMD ["streamlit", "run", "app.py", "--server.port=7860", "--server.address=0.0.0.0"]
```

Zero-Friction Demo Loading

App auto-discovers data and pre-computed artifacts on startup

```
# Auto-discovery candidates
LOCAL_SYNTHETIC_DATA = [
    os.getenv('VRP_SYNTHETIC_DATA_DIR', '').strip(),
    str(Path(__file__).parent / 'data'),
    str(Path(__file__).parent / 'synthetic_data'),
]

LOCAL_ARTIFACTS = [
    os.getenv('VRP_NOTEBOOK_ARTIFACTS_DIR', '').strip(),
    str(Path(__file__).parent / 'artifacts'),
]
```

What the user sees

- Open the app → CSVs load automatically
- Backend list shows '✓ found' for each
- Default radio = 'Load saved workbook'
- Click 'Build comparison' → instant results
- No upload required for demo
- Live solves still available with one click

🎥 Demo Video

15-minute walkthrough · all 6 tabs · AI Agent live

youtu.be/FqVuVjW20yo

Click the link to watch the full walkthrough

Key Design Decisions

Decision	Alternative	Why this
Gemini 2.0 Flash	Claude Sonnet, GPT-4o	Free 1500/day, no card, strong function-calling
Split-solve for OR-Tools EA	**Pin locked stops in model**	**Naïve pinning makes PATH_CHEAPEST_ARC infeasible**
Greedy fallback	Reject any-drop solutions	Guarantees 100% assignment, rarely triggers
Locked-prefix mechanism	Full re-solve per scenario	Preserves in-progress work, 11% faster
Haversine distance	OSRM for all backends	Public OSRM rate-limited; haversine = consistent
ortools_allow_drop = True	Hard-assign all orders	Disjunction prizes improve quality
18 fixed scenarios	User-defined mutations	Reproducible benchmarks, fair compare
Streamlit (not Gradio)	Gradio, Dash	Best multi-tab UX, native dataframe rendering

Roadmap

Near-term

- CI/CD with benchmark regression
- Persist OSRM cache to disk
- Multi-depot solving support
- SLA breach prediction model
- OpenAPI spec for headless solves
- Real-time technician GPS feed integration
- Add structured logging and request tracing

Long-term

- Anthropic Claude as alternative agent
- Kubernetes deployment manifest
- A/B testing for cost function variants
- Production load + concurrency tests
- Move to async backend execution
- Add per-customer custom scenarios
- Streaming agent responses

From solving a routing problem to shipping an autonomous, execution-aware platform.

Final Thought

The real value is not only the OR-Tools EA winner backend.

It's the comparative benchmark scaffolding, the locked-prefix mechanism, and the agentic AI layer that together make the system operationally trustworthy.

100%	0	770s	8	18	1
ASSIGNMENT	VIOLATIONS	RUNTIME	SOLVERS	SCENARIOS	AI AGENT

Thank you

Questions? Find everything here:

📦 Source Code

github.com/dranubhaparashar/Execution-Aware-Agentic-VRP-Solver-and-Benchmark-Studio

🚀 Live App

huggingface.co/spaces/AnubhaParashar/Execution-Aware-Agentic-VRP-Solver-and-Benchmark-Studio

🎥 Demo Video

youtu.be/FqVuVjW20yo

15-minute walkthrough

📚 Wiki Docs

Architecture · Algorithm Comparison · Function Reference